

Architectural Decision Record (ADR) – MS3 Media Service

Status: Aceito

Versão: 2.0

Data: Junho de 2026

Equipe: Grupo 9

2. Contexto

O sistema Plus necessita de um mecanismo para gerenciamento de imagens associadas a produtos e suas variações.

Um produto pode possuir múltiplas imagens e diferentes combinações de cor, tamanho ou grade. Algumas imagens podem representar o produto principal, enquanto outras podem estar associadas a variações específicas.

Além disso, as imagens precisam ser armazenadas de forma escalável e desacoplada do domínio de produtos, permitindo crescimento independente e reutilização por diferentes consumidores da plataforma. O projeto adota uma arquitetura baseada em microserviços, exigindo separação clara de responsabilidades e integração com os demais componentes do ecossistema.

3. O Problema

Como armazenar, organizar e disponibilizar mídias de produtos de forma:

- Escalável;
- Desacoplada do domínio de produtos;
- Integrada ao sistema de autenticação;
- Compatível com ambientes locais e cloud;
- Aderente aos requisitos da disciplina.

4. Decisão Arquitetural

Foi adotado um microsserviço independente denominado **MS3 – Media Service**, responsável exclusivamente pelo gerenciamento das mídias dos produtos.

Stack Tecnológica Adotada:

- **Framework:** FastAPI
- **Banco de Dados:** PostgreSQL
- **Autenticação:** JWT compartilhado com o MS Auth
- **Autorização:** RBAC (Role-Based Access Control - Admin/Vendedor)
- **Storage:** Armazenamento S3 compatível com AWS através do Ministack
- **Ambiente Local:** Docker Compose para orquestração

5. Responsabilidades do Domínio

O Media Service é responsável por:

- Upload e exclusão de mídias (arquivos físicos);
- Consulta de mídias e reordenação de exibição;
- Associação de mídias a produtos e suas variações específicas;
- Integração direta com o armazenamento de objetos S3.

6. Fora do Escopo

Não são responsabilidades deste microsserviço:

- Cadastro de produtos, variações ou controle de estoque;
- Processamento, edição de imagens ou conversão de formatos;
- Suporte a arquivos de vídeo;
- Autenticação de usuários e gestão centralizada de permissões.

Nota: Estas responsabilidades pertencem estritamente a outros componentes especializados da arquitetura do sistema Plus.

7. Modelo de Dados

Estrutura de metadados persistida na tabela do PostgreSQL:

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "id_produto": "Produto1",
  "id_variacao": "VariacaoA",
  "caminho_arquivo": "products/Produto1/variations/VariacaoA/550e8400-e29b-41d4-a716-446655440000",
  "ordem": 0,
  "data_criacao": "2026-06-20T10:00:00Z",
  "data_atualizacao": "2026-06-20T10:00:00Z"
}
```

O campo `caminho_arquivo` armazena a Object Key exata do arquivo dentro do bucket S3.

8. Integração com o MS Auth

O Media Service não realiza autenticação própria, delegando-a ao **MS Auth** por meio de tokens JWT.

Fluxo de Autenticação:

1. O usuário realiza login no **MS Auth**.
2. O MS Auth gera e assina um token JWT.
3. O JWT é enviado no Header das requisições direcionadas ao **MS Media**.
4. O MS Media valida a assinatura utilizando o mesmo `JWT_SECRET` compartilhado.
5. As permissões são verificadas localmente via RBAC.

Perfis e Regras Suportadas:

- **ADMIN:** Permissão total (Criar, remover e reordenar mídias).
- **VENDEDOR:** Acesso restrito apenas para leitura (Consulta).

9. Integração com o MS Produto

O Media Service depende da existência prévia de dados do domínio de produtos.

- Os identificadores `id_produto` e `id_variacao` são consumidos de forma externa.
- O Media Service atua de forma passiva, não criando, alterando ou removendo os produtos em si.

10. Integração com Armazenamento S3

Os arquivos binários não são armazenados no banco de dados. Foi adotado armazenamento compatível com AWS S3 utilizando o **Ministack**.

- **Bucket utilizado:** `plus-media`
- **Estrutura de chaves (Object Keys):**
 - *Produto principal:* `products/{id_produto}/media/{uuid}.jpg`
 - *Variação específica:*
`products/{id_produto}/variations/{id_variacao}/{uuid}.jpg`

11. Justificativa das Tecnologias

Tecnologia	Justificativa
FastAPI	Simplicidade, documentação Swagger automática e alta performance com suporte assíncrono.
PostgreSQL	Maturidade de mercado, confiabilidade transacional e integração robusta via SQLAlchemy.
JWT & RBAC	Autenticação descentralizada, garantindo baixo acoplamento e separação clara de papéis.
S3 / Ministack	Altamente escalável em relação ao disco local, permitindo fácil migração para a AWS Cloud.

12. Alternativas Consideradas

Alternativa 1: Imagens no Banco de Dados (BLOB)

Prós: Implementação inicial simples.

Contras: Crescimento excessivo e gargalo no banco, backups lentos e perda de performance.

Decisão: Rejeitada.

Alternativa 2: Imagens em Disco Local (Volume)

Prós: Simplicidade de escrita direta em sistema de arquivos.

Contras: Baixa escalabilidade horizontal, dificuldade de replicação e forte dependência do host.

Decisão: Rejeitada.

Alternativa 3: Object Storage (S3 API)

Prós: Isolamento completo de metadados e arquivos binários, aderente às práticas de cloud modernas.

Contras: Exige lógica adicional de sincronização com o banco.

Decisão: Adotada.

13. Trade-offs Arquiteturais

A adoção do S3 introduziu o desafio da consistência eventual: o banco de dados SQL e o Object Storage não participam da mesma transação atômica.

Para reduzir inconsistências sem adicionar a complexidade de transações distribuídas:

- Os uploads no S3 são removidos via rollback manual caso a gravação subsequente no PostgreSQL falhe.
- As exclusões removem o objeto físico do storage antes da deleção definitiva do registro no banco.

14. Consequências da Decisão

Positivas:

- Separação clara de responsabilidades entre serviços.
- Armazenamento e escalabilidade independentes do core do sistema.
- Arquitetura 100% aderente a ambientes reais AWS Cloud.

Negativas:

- Dependência de disponibilidade de um storage externo (ou mock local).
- Complexidade operacional ligeiramente maior (gestão de chaves, IAM e ambientes).

15. Validação da Solução

A solução foi homologada por meio dos seguintes critérios de qualidade:

- **Testes Manuais:** Executados de ponta a ponta via interface interativa do Swagger.
- **Integração Real:** Validação de persistência física no PostgreSQL e no container do Ministack S3.
- **Testes Automatizados:** 18 testes integrados aprovados rodando via Pytest.
- **CI/CD:** Pipeline automatizado executando a cada push no GitHub Actions.

16. Funcionalidade de Filtro por Variação

O Media Service suporta filtragem de mídias por variação de produto em todas as camadas da arquitetura.

Decisão:

O filtro foi implementado como **query parameter opcional** (`id_variacao`) no endpoint de consulta, preservando compatibilidade retroativa com clientes que não enviam o parâmetro.

Fluxo completo:

1. O usuário informa o `id_produto` e opcionalmente uma `id_variacao` (ex.: `cor-preta`) no MFE.
2. O cliente HTTP constrói a URL: `GET /products/{id_produto}/media?id_variacao=cor-preta` .
3. O controller FastAPI repassa o parâmetro ao `MediaService.get_media_by_product` .
4. O service adiciona o filtro na query SQLAlchemy apenas quando o parâmetro está presente, retornando somente as mídias da variação solicitada, ordenadas por `ordem` .

5. Quando nenhuma variação é informada, o comportamento é idêntico ao original (retorna todas as médias do produto).

Justificativa:

- Filtro no banco evita tráfego desnecessário de dados para o cliente.
- Query param é preferível a path param pois o filtro é opcional.
- Não requer migração de schema — o campo `id_variacao` já existia no modelo.

17. Integração com o Microfrontend e Shell App

O domínio de Mídia é exposto ao usuário por meio do **MFE plus-mfe-media**, construído com React + TypeScript + MUI, integrado ao **Shell App** via **Module Federation** (Vite Plugin Federation).

Decisões de integração cross-origin:

- O Shell App (`localhost:3000`) carrega o MFE de mídia (`localhost:4002`) como remote entry via `mfe_media/MediaDashboardPage` .
- Como `localStorage` não é compartilhado entre origens distintas, o token JWT é propagado via **query parameter** (`?token=...&refresh=...`) na navegação entre MFEs.
- O MFE de mídia executa um *bootstrap* ao inicializar: lê os parâmetros da URL, persiste em `localStorage` e `sessionStorage` , e limpa a URL via `history.replaceState` .
- Quando o usuário acessa `localhost:4002` sem token, é redirecionado ao login em `localhost:4001` com o parâmetro `?redirect=` , permitindo retorno automático com token após autenticação.

Justificativa da abordagem:

A passagem de token via URL foi adotada como alternativa a cookies (`SameSite` bloquearia cross-origin) e `postMessage` (exigiria acoplamento explícito entre MFEs). A URL é limpa imediatamente após o bootstrap, eliminando exposição prolongada do token.

17. Histórico de Revisões

- **v1.0 (Junho/2026):** Modelagem e definição inicial do escopo do domínio.

- **v2.0 (Junho/2026):** Atualização do documento refletindo a arquitetura final com segurança JWT/RBAC aplicada, testes automatizados e esteira de CI/CD integrada.